

| 풀스택 개발자 양성과정



프로그래밍 언어 활용

1조 정승원, 김선길, 박형규, 박한결

목차

INDEX

- | | | |
|----|------------|---|
| 01 | 별찍기 | 입력받은 높이만큼 삼각형, 다이아몬드 형태의 별을 찍어내는 프로그램 구현 |
| 02 | 사과장수 로직 구현 | 구매자와 판매자 클래스를 만들어 사과를 구매하고 금액을 차감하는 로직 구현 |
| 03 | 클로저 완성 | 코드에서 빠진 부분을 구현해 클로저 완성 |



01 별찍기(삼각형)

제시 문제 사진

| 문제

높이를 입력받아 높이만큼 삼각형을 찍어내는 프로그램을 구현합니다.

높이 입력 : 6

```
*  
***  
*****  
*****  
*****  
*****  
*****
```

01 | 삼각형 별 찍기

```
Scanner sc = new Scanner(System.in);
int n = sc.nextInt();
int i = 0;
while (i < n) {
    int j = 0;
    while (j < n - (i+1)) {
        System.out.printf(" ");
        j++;
    }
    int k = 0;
    while (k < 2*i + 1) {
        System.out.printf("*");
        k++;
    }
    System.out.println();
    i++;
}
sc.close();
```

1. 입력받은 숫자'n'만큼의 층을 가진 삼각형 모양의 별 삼각형을 출력하는 반복문 코드입니다.
2. 각 줄마다 공백은 점점 줄어듦, 별(*)의 개수는 홀수($2i+1$ 개)로 늘어나도록 하였습니다.
3. Scanner를 통해 정수를 입력받아 외부 while문으로 줄을 바꾸고, 내부 while문들로 공백과 별을 알맞게 배치하였습니다.



n=7 일 때 출력결과

```
7
      *
     ***
    *****
   *********
  **********
 ***
*****
*****
*****
```




제시 문제 사진

01 별찍기(다이아몬드)

문제

높이를 입력받아 높이만큼 다이아몬드(높이(h))는 홀수값만 입력받습니다)를 찍어내는 프로그램을 구현합니다.

```
높이 : 9
  *
 ***
*****
*****
*****
*****
*****
  ***
   *
```

01 | 다이아몬드 별 찍기

```
Scanner sc = new Scanner(System.in);
int h = sc.nextInt();
int i = 0;
int l = 0;
while (i < h) {
    int j = 0;
    while (j < (h/2) - 1) {
        System.out.printf(" ");
        j++;
    }
    int k = 0;
    while (k <= l*2) {
        System.out.printf("*");
        k++;
    }
    if (i < (h/2)) {
        l++;
    }
    else l--;

    System.out.println();
    i++;
}
sc.close();
```

1. 다이아몬드 형태의 별 패턴을 출력하는 코드로, 입력값(h)에 따라 전체 높이가 결정됩니다.
2. 변수 l을 이용해 가운데 줄까지는 별이 늘어나고 (공백 감소), 그 이후부터는 다시 줄어들도록 제어하는 것이 핵심입니다.
3. if문을 통해 상단과 하단의 분기점을 나누어 공백과 별의 개수를 유동적으로 조절하여 모양을 완성합니다.



n=7 일 때 출력결과



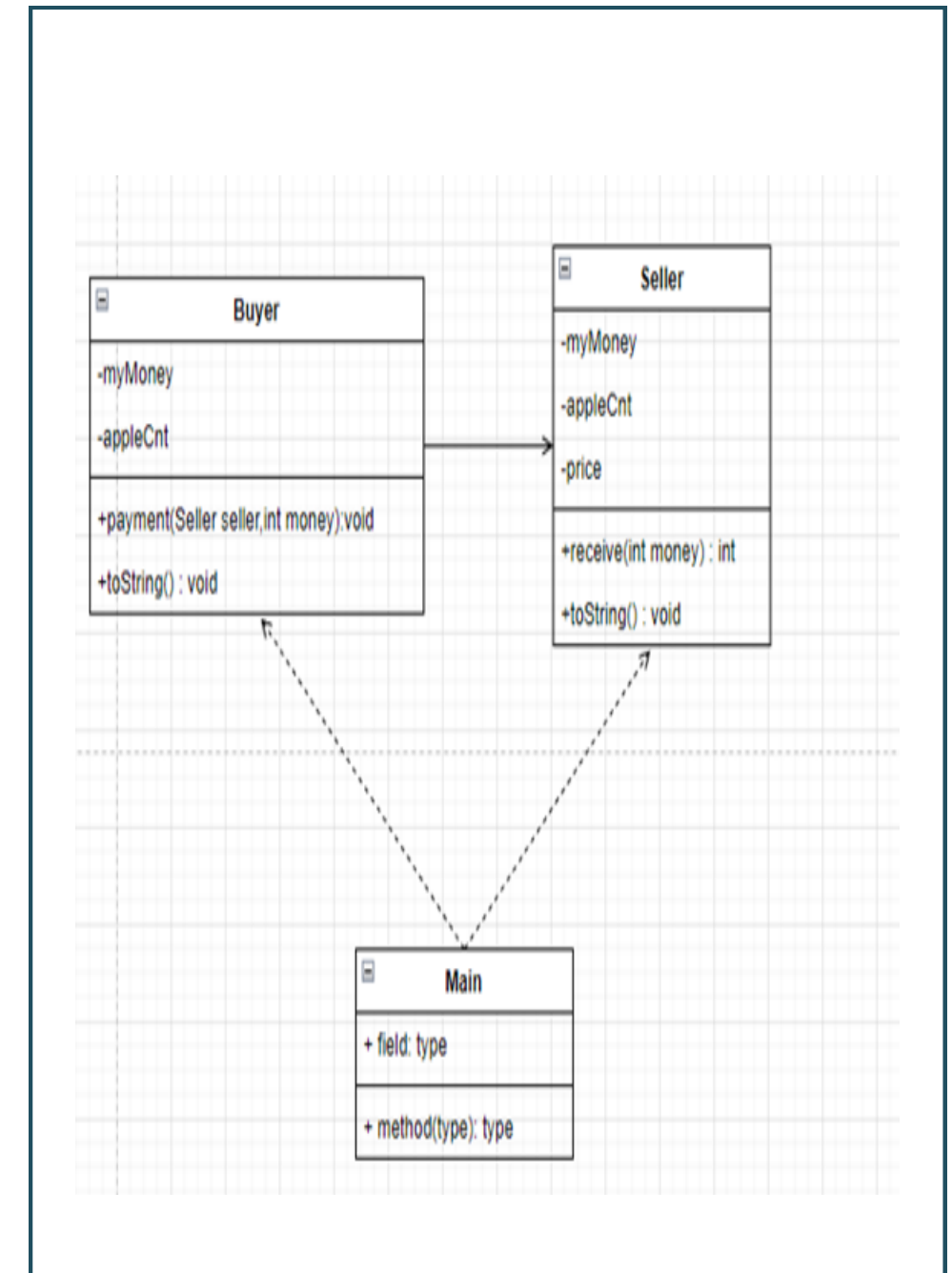


02 사과장수

제시 문제 사진

문제

다음 클래스 다이어그램을 참고해서 Buyer Class, Seller Class 구현합니다.



02 사과장수 Buyer

```
class Buyer {
    private int myMoney;
    private int appleCnt;

    // 생성자
    public Buyer(int myMoney, int appleCnt) {
        super();
        this.myMoney = myMoney;
        this.appleCnt = appleCnt;
    }

    public void pay(Seller seller, int money) {

        // 판매자에게 구매의사금액 전달
        // 판매자는 구매자에게 사과개수 전달
        // 내 돈에서 구매의사금액만큼 차감 (내보유금액 > 구매의사금액)
        myMoney -= money;
        int receivedAppCnt = seller.sales(money);
        // 사과개수를 현재 사과보유개수에 누적
        this.appleCnt += receivedAppCnt;
    }

    // toString
    @Override
    public String toString() {
        return "Buyer [myMoney=" + myMoney + ", appleCnt=" + appleCnt + "];"
    }
}
```

1. Buyer 클래스는 구매자의 보유 금액(myMoney)과 사과 개수(appleCnt)를 관리하는 역할을 합니다.
2. pay() 메서드는 판매자(Seller)에게 구매 금액을 전달하고, 받은 사과 개수를 구매자의 사과 개수에 추가합니다. 또한 보유 금액이 부족하면 가진 돈만큼만 구매하도록 처리합니다.
3. toString() 메서드를 오버라이딩하여 구매 후 구매자의 현재 돈과 사과 개수를 문자열 형태로 출력할 수 있게 합니다.

02 | 사과장수 Seller

```
class Seller {
    private int myMoney;
    private int appleCnt;
    private int price;

    public int sales(int money) {
        // 구매자로 부터 받은 금액을 내 보유금액에 누적
        this.myMoney += money;
        // 구매자로 부터 받은 금액만큼 사과개수를 계산
        int calAppCnt = money / price;
        // 판매자의 보유 사과개수에서 차감(보유사과개수 vs 구매자 요청사과개수)
        this.appleCnt -= calAppCnt;
        // 계산된 사과 개수를 반환
        return calAppCnt;
    }
    public Seller(int myMoney, int appleCnt, int price) {}
}
@Override
public String toString() {
    return "Seller [myMoney=" + myMoney + ", appleCnt=" + appleCnt + ", price=" + price + "];"
}
}
```

1. Seller 클래스는 판매자의 보유 금액(myMoney), 사과 재고(appleCnt), 사과 가격(price)을 관리하는 역할을 합니다.

2. sales() 메서드는 구매자로부터 받은 돈을 판매자의 금액에 추가하고, 금액에 맞는 사과 개수를 계산하여 재고에서 차감한 뒤 반환합니다.

3. 생성자를 통해 초기 자금, 사과 개수, 가격을 설정하며, toString() 메서드로 판매자의 현재 상태를 문자열 형태로 출력합니다.

02 | 사과장수 Buyer, Seller

```
public class C12Ex_중요 {  
    public static void main(String[] args) {  
        // 고객  
        Buyer 홍길동 = new Buyer(100000, 0);  
        Buyer 남길동 = new Buyer(100000, 0);  
  
        // 사과장수  
        Seller 사과장수1 = new Seller(100000, 100, 1000);  
  
        // 고객 - 구매요청 -> 사과장수  
        홍길동.pay(사과장수1, 20000);  
        남길동.pay(사과장수1, 10000);  
  
        System.out.println(홍길동);  
        System.out.println(남길동);  
        System.out.println(사과장수1);  
    }  
}
```

1. Buyer와 Seller 객체를 생성하여 고객과 사과장수의 초기 자금 및 재고 상태를 설정합니다.
2. pay() 메서드를 사용해 고객이 사과장수에게 돈을 지불하고, 금액에 맞는 사과를 구매하도록 처리합니다.
3. 거래가 끝난 뒤 System.out.println()으로 각 객체 정보를 출력하여 구매 후 변경된 상태를 확인합니다.



출력결과

```
Buyer [myMoney=80000, appleCnt=20]  
Buyer [myMoney=90000, appleCnt=10]  
Seller [myMoney=130000, appleCnt=70, price=1000]
```

홍길동 → 돈 100000, 사과 0
남길동 → 돈 50000, 사과 0
사과장수 → 돈 100000, 사과 100, 가격 1000

홍길동이 20000원 구매:

사과 개수 = $20000 / 1000 = 20$
홍길동 돈 → 80000
홍길동 사과 → 20
사과장수 돈 → 120000
사과장수 사과 → 80

남길동이 10000원 구매:

사과 개수 = $10000 / 1000 = 10$
남길동 돈 → 40000
남길동 사과 → 10
사과장수 돈 → 130000
사과장수 사과 → 70



03 클로저(closure)

제시 문제 사진

문제

다음 코드를 확인하여 클로저를 완성하세요.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Document</title>
</head>
<body>
  <script>
    function createCounter() {
      let count = 0;

      function increment() {
        // 코드입력(포함될 때마다 count에 1씩 증가)
        return count;
      }
      return ; //코드입력 (increment함수를 반환)
    }

    const counter = createCounter();
    console.log(counter()); // 출력: 1
    console.log(counter()); // 출력: 2
    console.log(counter()); // 출력: 3
  </script>
</body>
</html>
```

DevTools is now available in Korean!

Always match Chrome's language

Switch DevTools to Korean

Don't show again

Elements Console Sources Network Performance >> ⚙️ ⋮ ✕

🔍 🔍 top 🔍 Filter Default levels No Issues ⚙️

1	index.html:22
2	index.html:23
3	index.html:24
>	

03 | 클로저(closure)

```
<script>
function createCounter() {
  let count = 0;

  function increment() {
    //코드입력 (호출될때마다 count 1씩 증가)
    ++count;
    return count;
  }
  return increment; // 코드입력 (increment함수를 반환)
}

const counter = createCounter();

console.log(counter()); // 출력 :1
console.log(counter()); // 출력 :2
console.log(counter()); // 출력 :3
</script>
```

1. increment() 함수 본문 안에 ++count;를 넣어 increment 함수가 호출될 때마다 count를 1씩 증가하게 만듭니다.

2. return 반환값에 increment를 넣어 increment 함수를 반환시킵니다.



출력결과

